

포인터

자료형 *변수명;

```
#include<stdio.h>
int main()
{
    int a = 7;
    int* ptr = &a;

    printf("a의 주소값 : %p\n", &a);
    printf("a의 값 : %d\n\n", a);

    printf("ptr의 주소값 : %p\n", &ptr);
    printf("ptr의 값 : %p\n", ptr);
    printf("ptr이 가르키는 주소의 값 : %d\n\n", *ptr);

    a = 3;
    printf("a의 값 변경 후, ptr이 가르키는 주소의 값 : %d", *ptr);
    return 0;
}
```

output:

```
a의 주소값 : 000000000061FE1C
a의 값 : 7

ptr의 주소값 : 000000000061FE10
ptr의 값 : 000000000061FE1C
ptr이 가르키는 주소의 값 : 7
```

변수명 : a
주소값 : 000000000061FE1C
값 : 7

변수명 : ptr
주소값 : 000000000061FE10
값 : 000000000061FE1C

※ 포인터 변수는 오직 다른 주소값을 담고 있을 뿐이다.

포인터와 배열의 연관성

```
#include <stdio.h>
int main()
{
    int arr[3] = { 5, 2, 3 };
    // 배열은 포인터 변수다?
    printf("arr의 주소값 %p\n", arr);
    printf("arr[0]의 주소값 %p\n", &arr[0]);
    printf("arr[1]의 주소값 %p\n", &arr[1]);
    printf("arr[2]의 주소값 %p\n", &arr[2]);
    printf("arr 역참조 %d\n\n", *arr);

    // 배열의 이름은 포인터 상수
    int num1 = 3, num2 = 5;
    int* ptr = NULL;
    ptr = &num1;
    ptr = &num2;
    // arr[1]=&num1; => 배열에는 주소값 저장이 안됨 => 배열은 포인터 변수라기보단 포인터 상수에 가깝다.

    // arr+1과 &arr+1의 값은 같지 않다.
    printf("arr+1 : %p != &arr+1 : %p\n\n", arr + 1, &arr + 1);
    // arr+1은 4바이트, &arr+1은 12바이트 더 커졌다.

    // arr+1의 주소값과 a[1]의 주소값은 같다.
    printf("arr : %p == &arr[0] : %p\n", arr, &arr[0]);
    printf("arr+1 : %p == &arr[1] : %p\n", arr + 1, &arr[1]);
    printf("arr+2 : %p == &arr[2] : %p\n", arr + 2, &arr[2]);
    return 0;
}
```

output:

```
arr의 주소값 000000000061FE0C
arr[0]의 주소값 000000000061FE0C
arr[1]의 주소값 000000000061FE10
arr[2]의 주소값 000000000061FE14
arr 역참조 5

arr+1 : 000000000061FE10 != &arr+1 : 000000000061FE18

arr : 000000000061FE0C == &arr[0] : 000000000061FE0C
arr+1 : 000000000061FE10 == &arr[1] : 000000000061FE10
arr+2 : 000000000061FE14 == &arr[2] : 000000000061FE14
```

배열은 오직 첫 번째 위치의 주소를 가지고 있다.

따라서, 배열의 다른 인덱스의 값들은 첫 번째 위치의 주소에 더해서 접근한다.

동적 할당

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    //malloc/ calloc 으로 동적할당
    //malloc은 0으로 초기화 안하지만 calloc은 전부 0으로 초기화 해줌
    int* maloc = (int*)malloc(sizeof(int) * 2);
    int* caloc = (int*)calloc(2, sizeof(int));
    for (int i = 0; i < 2; i++) {
        printf("malloc : %d  calloc : %d\n", maloc[i], caloc[i]);
    }

    *maloc = 3;
    *(maloc + 1) = 8;

    printf("%d %d\n", *maloc, *(maloc + 1));

    *(maloc + 2) = 2;
    realloc(maloc, sizeof(int) * 2);

    for (int i = 0; i < 4; i++) {
        printf("%d\n", *(maloc + i));
    }

    //메모리 해제
    free(maloc);
    free(caloc);
    return 0;
}
```

output:

```
malloc : 13132080  calloc : 0
malloc : 0  calloc : 0
3 8
3
8
2
0
```

동적 할당은 다른 변수들을 선언하는 것과 달리, 컴파일 단계가 아니라, 실행 단계에서 메모리를 할당 받는 방법이다.

다른 정적 배열 또는 변수들은 실행할 때, 크기가 정해지므로 수정이 불가능하다.

하지만, 동적 할당을 이용하면 프로그램 실행 도중에 크기를 수정할 수 있다.

동적 할당으로 메모리를 할당받아 나누어서 사용하면 배열이 되고, 한 번에 쓰면 변수가 된다고 생각하면 편하다.